sampling parameters: (a) temperature $T \in \{0.5, 1.0\}$, and (b) top-$p \in \{0.9, 1.0\}$. We used only one candidate for P_{task} i.e., "`Please read the below problem description and generate a python code to solve the problem {problem description} Please only generate code and nothing else.`" While we experimented with providing the LLM with a persona i.e., asking the LLM to pretend to be a "grandmaster in solving competitive programming problems", the resulting accuracy was lower (see Appendix E.2). For each combination of LLM, sampling parameter, and prompt, we produce $\forall i, m_i \geq 20$ generations such that at least 20 of the generated solutions were correct (details in Appendix D.2). We instantiated this by generating k samples ($k = 100$ for `GPT-4` and $k = 200$ for `Claude-3`), and verifying that at least 20 of them were correct.

Attribute Extraction: We consider the following attributes which characterize different aspects of code. We rely on `GPT-3.5` for extracting some of the attributes; we manually verified the extracted attributes and confirmed their quality is high (see Appendix E.3).

1. *Correctness* refers to whether a piece of code correctly solves the given problem and passes all the test cases. We measure accuracy as the ratio of correct solutions among all solutions (details in Appendix D.3), to quantify the quality of human-/model-generated solutions.
2. *Efficiency* is crucial for scalability [44]. This is measured through: asymptotic time/space complexity and runtime efficiency. We prompt `GPT-3.5` to infer the big O time and space complexity [45], and execute the code on test cases to measure runtime and memory usage (see Appendix D.4).
3. *Fingerprint* provides insights into the structural and syntactical uniqueness of each code segment. As stated in Section 4.3, we use the COPYDETECT tool for this.
4. *Code Summary (textual)* explains the functionality of the code. Prior work has demonstrated the effectiveness of `GPT-3.5` in code understanding [46]. Thus, we use it to produce text-based summaries, and a `description`, `functionality`, `algorithm`, and `data structure` (prompt for this task is in Appendix D.5). To compare the similarity for these text summaries, we produce their embeddings using the `all-MiniLM-L6-v2` model [47].
5. *Code Summary (categorical)* reflects the techniques employed in the code through categorical tags, as used on the Codeforces website. We prompt `GPT-3.5` to assign `tags` to a code segment by providing it a set of tags to choose from (prompt for this task is in Appendix D.5). We obtain one set per code segment. We similarly prompt `GPT-3.5` to choose from a list of `algorithms` and `data structures`.

Metric Calculation: For correctness, we calculate the mean value i.e., accuracy over the conditional distribution. For efficiency (asymptotic complexity), we calculate: (a) entropy for the conditional distribution, and (b) plot the histogram for the unconditional distribution. For runtime efficiency, we calculate mean and standard deviation for the conditional distribution. We measure the following over the conditional distribution: fingerprints, where we calculate the mean pairwise fingerprint similarity; code summary (textual), where we calculate the mean pairwise cosine similarity in their embedding space, and code summary (categorical), where we calculate the mean pairwise Jaccard index.

7 Results and Takeaways

Guide: We present our results on measuring, and attempting to mitigate, generative monoculture in Fig. 4 and 5. We display results mainly in three formats: (a) stacked bar charts, where different hues correspond to different value ranges as indicated in the legend; (b) histograms (or grouped bar charts), to reflect the probability mass of a categorical variable; and (c) kernel density estimation (KDE) plots, to reflect the estimated probability density of a continuous variable. We note that, in the code results, for all plots except that evaluating accuracy, we restrict to correct solutions.

Takeaway 1: Monoculture Exists and is Severe, Within and Across LLMs. As shown in Fig. 4 and 5, there exists significant narrowing from the source to generation distribution in all attributes considered for both scenarios, book reviews and coding.

Notably, for book review, *proprietary OpenAI LLMs (`GPT-3.5` and `GPT-4`) demonstrate even more severe monoculture compared with the open-source `Llama` family LLMs* (see Fig. 9 in Appendix C.5 for more details). Particularly, for both `GPT-3.5` and `GPT-4`, 100% of the samples have average

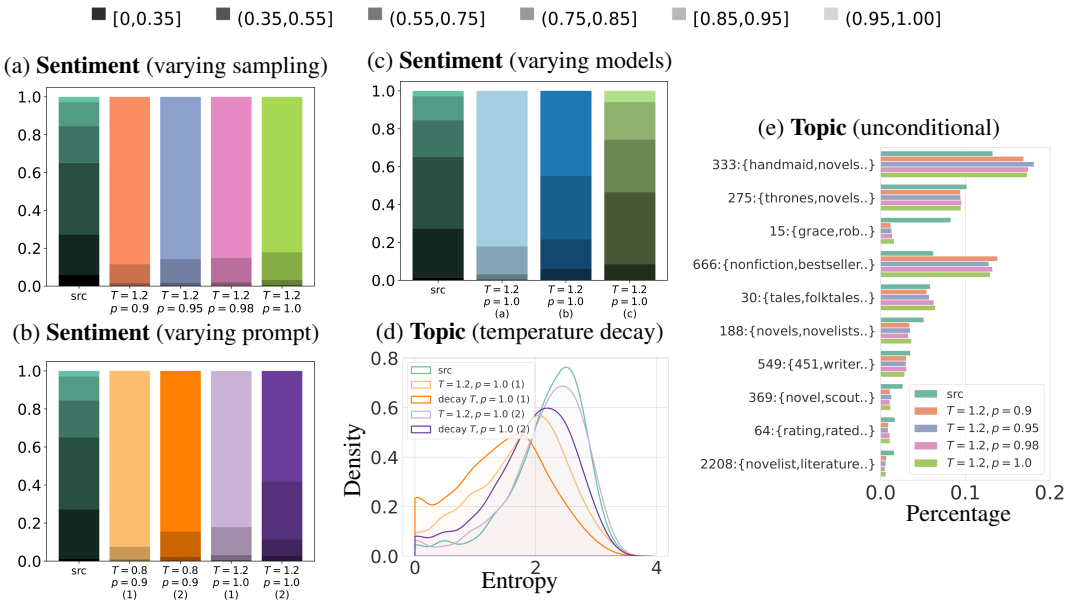

Figure 4: (**a-c**) stacked barplots for the mean sentiment scores under varying sampling parameters, prompts, and models. For these plots, in each bar, darker hues (bottom) represent lower scores while lighter one (top) denote higher scores. See the legend for the value range of each hue. In subfigure **(b)**, (1) and (2) refer to the two prompts as introduced in Section 6.1. In subfigure **(c)**, (a-c) refer to `Llama-2-chat`, `Vicuna-13b`, and `Llama-2`. **(d)** kernel density estimation (KDE) for the entropy values calculated on the conditional distribution of the topics. (**e**) unconditional topic distribution for top-10 topics. For all the subfigures, we mark the sampling parameters in them; unless marked with (1-2) or (a-c), the results are obtained on `Llama-2-chat` under prompt (1). These subfigures show that the model-generated reviews are overwhelmingly positive and cover a narrower range of the topics per book; moreover, there is distinctive under- and over-representation of the topics covered overall.

positivity falling in (0.95,1.00] under prompt (1), and 98.9% and 97.6% under prompt (2). For coding, similar reductions in diversity can be seen: in Fig. 5(e), we see increased similarity in natural language descriptions of LLM-generated code solutions, and Fig. 5(f) shows the Jaccard similarity of the generated solutions in terms of the inferred `algorithms`, with the majority of problems displaying high similarity across generated solutions. Of particular interest, the plagiarism scores of the LLM-generated code are extremely high (Fig. 5(b)), compared to the source solutions which achieve an utterly zero plagiarism score for *all* the problems. We examine a few pairs of examples and their plagiarism scores in Appendix E.1.

Takeaway 2: LLMs tend to produce Human-favorable Generations. Our results show that LLMs tend to over-represent parts of the attribute distribution that are preferred by humans: humans largely prefer text with positive sentiment, as well as correct and efficient code. Fig. 4(a) and Appendix C.5 show that LLMs produce overwhelmingly positive generations. Fig. 5, as well as Fig. 21 and 22 in the Appendix reveal that LLM-generated code segments (a) are over $2\times$ more accurate than the average human solutions, (b) enjoy an overall lower asymptotic time and space complexity, and (c) use less runtime and memory during execution. This may just be the intended consequence of RLHF, which explicitly optimizes the LLM towards producing human-favored responses in its objective, as guided by a reward model trained on human preferences.

However, as our results show, this implies a loss of diversity guided by human preferences, which, if only naively understood and enforced, could lead to unwanted consequences if going unnoticed. One example of the unintended artifacts is the under- and over-represented topics (Fig. 4(e)); the topic group 15 which contains keywords "rob" and "kill" etc. is significantly under-represented, likely a consequence of RLHF alignment tuning.

Takeaway 3: RLHF Hurts Diversity the Most. `Llama-2-chat` is obtained via performing RLHF tuning [17] on the pre-trained (PT) `Llama-2`. Similarly, `Vicuna-13b` is obtained via supervised fine-tuning (SFT) on the PT `Llama-2` [33]. Comparisons on these LLMs (see Fig. 4(c), as well

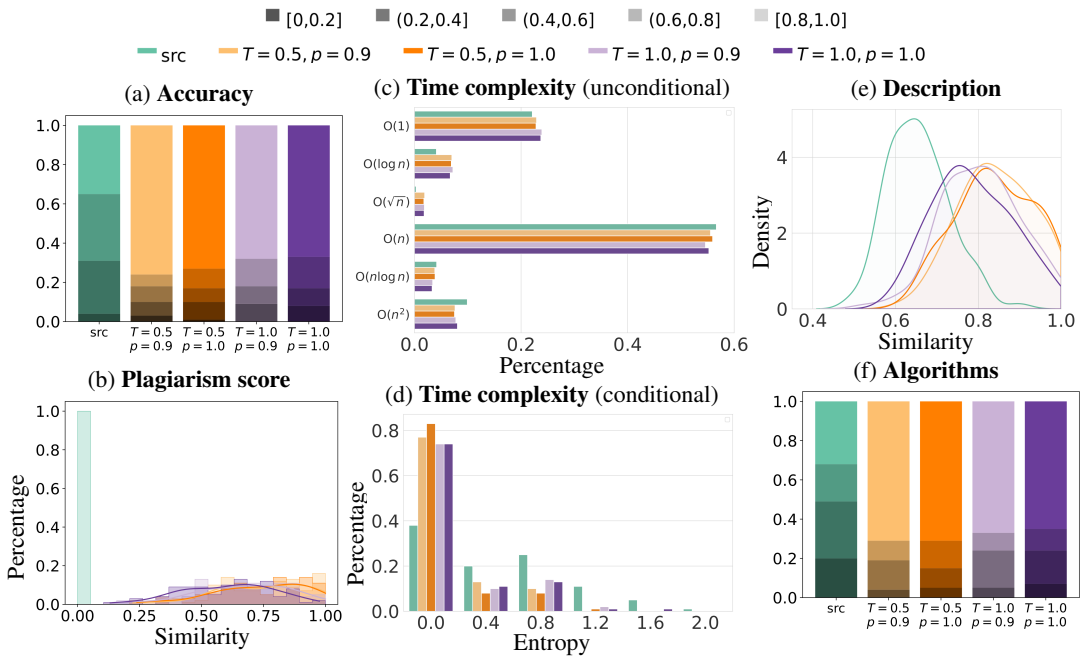

Figure 5: (**Left**) **(a)** stacked barplot for accuracy and **(b)** probability mass along with KDE for plagiarism scores. (**Middle**) Time complexity: **(c)** histogram of the (unconditional) distribution of the asymptotic complexity and **(d)** probability mass for the (conditional) distribution of entropy values. (**Right**) Selected code summary: **(e)** KDE plot for the mean pairwise cosine similarity scores for "description" as natural language and **(f)** stacked barplot for the mean pairwise Jaccard scores for "algorithms" as categorical values. Overall, the model-generated solutions are more accurate and efficient, display higher description similarity to each other, and cover a narrower span of algorithms. (More results in Appendix E.5.)

as Fig. 8 in Appendix C.4) show that the PT LLM-generated reviews are much more similar to the source. The PT LLM `Llama-2` has 5.9% of samples with average sentiment values falling in the range of (0.95,1.00], which is much closer to the source percentage of 2.8% than 44.7% for `Vicuna-13b` and 82.1% for `Llama-2-chat`. `Vicuna-13b` also shows better diversity than `Llama-2-chat`—this is consistent with findings suggesting RLHF reduces output diversity compared with SFT (albeit with different metrics) [48].

Takeaway 4: Naive Mitigations are Insufficient. Changing the sampling parameter (increasing T and p) and using a more diversity-inducing prompt (e.g., prompt (2) for book reviews) can reduce the gap (see Fig. 4(a-b) and Fig. 5). For example, using prompt (2) reduces the percentage of the most positive range from 82.1% to 58.1% in Fig. 4(b) for $T = 1.2, p = 1.0$. However, the gap is still large. More results in Appendix (Figures 12 and 13, and Appendix E) show similar conclusions.

Table 1: **The average fraction of valid generations (out of a total of 10)** for two models under various sampling parameters (temperature, top-p, and prompt–denoted (1) and (2)). We regard a "valid" generation as text of perplexity value ≤ 20—as a support, we present high perplexity samples in Appendix C.1. We observe that the number drops as the randomness increases (along the increase of both T and top-p values). As a reference, `GPT-4` achieves an average ratio of 1.000 at $T = 1.2$ and $p = 1.0$.

LLama-2 -13b-chat	$p = 0.90$		$p = 0.95$		$p = 0.98$		$p = 1.00$		**Vicuna -13b**	$p = 0.90$		$p = 0.95$		$p = 0.98$		$p = 1.00$	
	(1)	(2)	(1)	(2)	(1)	(2)	(1)	(2)		(1)	(2)	(1)	(2)	(1)	(2)	(1)	(2)
$T = 0.5$	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	$T = 0.5$	0.987	0.992	0.987	0.990	0.984	0.989	0.984	0.988
$T = 1.0$	1.000	1.000	0.999	1.000	0.999	1.000	0.998	0.999	$T = 1.0$	0.961	0.962	0.951	0.958	0.942	0.953	0.935	0.947
$T = 1.5$	0.994	0.988	0.930	**0.883**	**0.743**	**0.649**	**0.512**	**0.394**	$T = 1.5$	**0.871**	0.903	**0.835**	**0.876**	**0.766**	**0.840**	**0.680**	**0.767**

We attempted two other strategies: (a) picking a higher temperature, and (b) leveraging a decaying temperature scheme (see § 5). Results in Appendix C.2 show that the gap still remains big even at such high randomness. Furthermore, for larger T, we notice a significant degradation of the generation quality as a result of the increased randomness. In Table 1, we present the average fraction of valid generations for `Llama-2-chat` and `Vicuna-13b` under various sampling parameters. The